

PARA ESTUDIO INDEPENDIENTE FUERA DE CLASE

8 – Comprendiendo las instrucciones de Carga

Revise el video explicativo (<https://youtu.be/38WJtX0c4A>). Las arquitecturas RISC habitualmente se denominan arquitecturas LOAD/STORE (Carga/Almacenamiento) dado a que todas las instrucciones, como las realizadas por la ALU, solamente pueden tomar como argumentos valores almacenados en el Register File (Formato de instrucción R) o números inmediatos (Formato de Instrucción I). En las arquitecturas RISC no se permite operaciones en la ALU con valores almacenados en la memoria.



Por ejemplo, supongamos que en memoria se tienen 3 variables X, Y y Z, ubicadas en las posiciones 0x10, 0x14, 0x18, respectivamente, y se quiere hacer la operación $Z = X + Y$. Entonces, para poder implementar este algoritmo, es necesario cargar (LOAD) las variables X y Y en registros temporales al interior del Register file, por ejemplo, x5 y x6; luego sumar los registros x5 y x6 y almacenarlos en otro registro temporal, por ejemplo, x7, y el resultado final, se debe almacenar (STORE) de nuevo en la memoria:

```
lw x5,0x10(x0)    #Carga la posición de memoria 0x10 en el registro x5
lw x6,0x14(x0)    #Carga la posición de memoria 0x14 en el registro x6
add x7,x5,x6       #x7 = x5 + x6
sw x7,0x18(x0)    #Almacena en la posición de memoria 0x18 el registro x7
```

Para verificar el resultado de este programa en el simulador <https://www.cs.cornell.edu/courses/cs3410/2019sp/riscv/interpreter/> es necesario inicializar la memoria externa por medio de instrucciones, colocando las siguientes instrucciones resaltadas en verde:

```
addi x5,x0,12      #Carga el registro x5 con el número 12
sw x5,0x10(x0)     #Almacena el número 12 en la posición de memoria 0x10 (variable X)
addi x5,x0,23      #Carga el registro x5 con el número 23
sw x5,0x14(x0)     #Almacena el número 12 en la posición de memoria 0x14 (variable Y)

lw x5,0x10(x0)     #Carga la posición de memoria 0x10 en el registro x5
lw x6,0x14(x0)     #Carga la posición de memoria 0x14 en el registro x6
add x7,x5,x6       #x7 = x5 + x6
sw x7,0x18(x0)     #Almacena en la posición de memoria 0x18 el registro x7
```

Preguntas:

- ¿Con qué valor queda la variable Z (posición de memoria 0x18) después de ejecutar el código anterior?
- En el código anterior las direcciones se escribieron con un formato del tipo 0x10(x0), es decir, que la dirección desde donde se carga o almacena el dato es $x0 + 0x10$. ¿Qué pasaría si en lugar de usar x0 se hubiera puesto x1, y x1 se hubiera cargado previamente con el número 4?
- Revise las instrucciones de carga en La Guía Práctica de RISC-V (pg. 4, <http://www.riscvbook.com/>). ¿Qué diferencia habrá entre las instrucciones LB, LH, LW?

9 – Modos de direccionamiento indirecto

En los programas ejemplo de la sección anterior se han usado las referencias a la memoria del tipo `DireccionImm(x0)`, pues esto asume que los datos se cargarán de la posición `DirecciónFinal = DireccionImm + x0 = DireccionImm`, ya que el registro `x0` siempre vale 0.

Los programadores o compiladores explotan esta propiedad del conjunto de instrucciones para implementar lo que se denomina **direccionamiento indirecto**, donde se usa un registro como puntero a una zona de memoria. Por ejemplo, suponga que la memoria está cargada con los siguientes números y se ejecuta el siguiente programa:

Posición de memoria	Dato
0x10	5
0x14	-1
0x18	2
0x1C	3



```
add x5, x0, x0
addi a4, x0, 0x10
lw x6, 0(a4)
add x5, x5, x6
addi a4, a4, 4
lw x6, 0(a4)
add x5, x5, x6
addi a4, a4, 4
lw x6, 0(a4)
add x5, x5, x6
addi a4, a4, 4
lw x6, 0(a4)
add x5, x5, x6
```



En este ejemplo, se ha usado un alias del registro `x14` llamado `a4`.

- ¿Qué ecuación está implementando este programa? ¿Es decir con que valor queda al final el registro `x5`?
- ¿Qué función cumple el registro `a4`?
- ¿Qué función cumple las instrucciones `addi a4, a4, 4` que están intercaladas en el código?



PARA EL LABORATORIO

En RISC-V la memoria de datos y programa se direcciona en bytes, pero las instrucciones de carga y almacenamiento operan sobre 32bits (4bytes). Por esta razón, para no añadirle complejidad a nuestra CPU, asumiremos que la información en la memoria de datos está alineada 4 bytes, es decir que los datos están almacenados en las direcciones, `0x0`, `0x4`, `0x8`, `0xC`, etc., y no se permiten direcciones como `0x1`, `0x2`, `0x3`, `0x5`, `0x6`, etc.

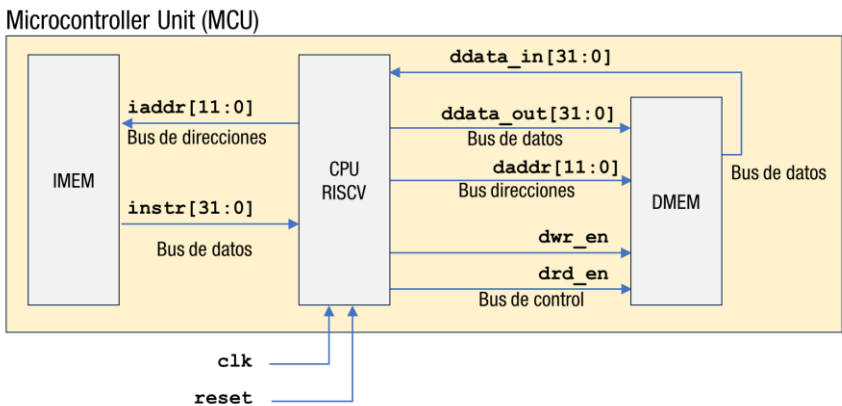


Figura 15. Nuestra implementación para el laboratorio del sistema mínimo basado en procesador RISC-V

Esta alineación nos permite añadir a nuestro microcontrolador una memoria de datos (DMEM) con un bus de datos de 32bits, como se muestra en la Figura 15, y para generar el bus de direcciones de datos (daddr) ignoraremos los dos bits menos significativos de la dirección de carga/almacenamiento. Más adelante se mostrará como calcular esta dirección.

Como se ilustra en la Figura 15, necesitamos añadir una memoria de datos DMEM a nuestro microcontrolador, lo cual debe hacerse en el archivo donde creó el módulo mcu. Debemos añadir una memoria SRAM, a la que llamaremos DMEM, la cual debe cablearse como se muestra en la Figura 15. Este cableado se muestra en el siguiente código. **No olvide que debe crear con una IA la memoria SRAM con la especificación dada en la Figura 15.** Para el prompt de la IA debe identificar de la Figura 15, el número de posiciones de memoria (en palabras, words, no bytes), tener en cuenta que el bus de datos está separado en dos, bus de datos de escritura y bus de datos de lectura, y que al existir señal de habilitación de lectura (drd_en), la salida debe ser Hi-Z cuando esta señal está inactiva. Esto es importantísimo ya que más adelante conectaremos más hardware a este bus de datos y debemos evitar cortocircuitos.

```
1 //Definición del microcontrolador
2 module mcu(
3     input clk, //Reloj del microcontrolador
4     input rst //Reset del microcontrolador
5 );
6
7 ... //Use la misma declaración de señales e instanciación de la CPU RISC-V e IMEM
8
9 //Instancia de la memoria de datos
10 sram_32x DMEM(
11     .clk(clk),
12     .addr(daddr[XXX:2]), //Debe cambiar XXX por el número correcto a su diseño
13     .wr_en(dwr_en),
14     .wr_data(ddata_out),
15     .rd_en(drd_en),
16     .rd_data(ddata_in)
17 );
18
19 endmodule
20
21 //Memoria ROM
22 module rom4096x32 (
23     //... Haga un refactoring de la ROM de la guía anterior para que sea sincrónica
24 endmodule
25
26 //Pon aquí la declaración de la memoria SRAM de datos
27 //...
28 module sram_32x (
29     ...
```

Una vez creada y conectada la memoria, necesitamos modificar la CPU RISC-V para detectar las señales de carga y almacenamiento y con base en ellas generar las señales **daddr**, **dwr_en**, **ddata_out**, **ddata_in**, **drd_en**.

Dado que nuestro microcontrolador será probado sobre una FPGA, es necesario garantizar que el diseño sea compatible con dicha tecnología, por ello es necesario que todas las memorias, tanto la RAM de datos (DMEM) como la ROM de programa (IMEM) sean declaradas sincrónicas, es decir, dependan de la señal de reloj. En ese sentido, es necesario hacer un refactoring de la memoria de programa para que sea sincrónica.

Al igual que hemos hecho en guías anteriores, se muestra a continuación el formato de instrucción de las instrucciones de carga y almacenamiento (página 44 de la guía RISC-V).

31	25	24	20	19	15	14	12	11	7	6	0	
imm[11:0]				rs1	000		rd		0000011			I lb
imm[11:0]				rs1	001		rd		0000011			I lh
imm[11:0]				rs1	010		rd		0000011			I lw
imm[11:0]				rs1	100		rd		0000011			I lbu
imm[11:0]				rs1	101		rd		0000011			I lhu
imm[11:5]		rs2		rs1	000		imm[4:0]		0100011			S sb
imm[11:5]		rs2		rs1	001		imm[4:0]		0100011			S sh
imm[11:5]		rs2		rs1	010		imm[4:0]		0100011			S sw

Note que las instrucciones de carga (load) (LB, LH, LW, LBU, LHU) usan el formato de instrucción I. Por simplicidad, nuestra CPU RISC-V, al usar datos alineados, solamente trabajará con palabras (Word), por lo que simplificaremos su diseño detectando todas estas instrucciones de carga como una sola. Incluya en la decodificación de instrucciones la señal **is_load** que se active cuando se detecta LB, LH, LW, LBU o LHU.

Para las instrucciones de almacenamiento (store) (SB, SH, SW), no es necesario incluir ninguna lógica de decodificación adicional, ya que son las únicas instrucciones que usan el formato de instrucción S, de allí que usaremos la señal **is_s_instr**, creada anteriormente, para referirnos a cualquiera de las instrucciones SB, SH y SW.

Para implementar estas instrucciones revisemos cómo funcionan:

Load (LW,LH,LB,LHU,LBU). Su formato es el siguiente y usa instrucciones del tipo I:

LOAD rd, imm(rs1)

A nivel funcional, el registro de destino (rd) se carga con el valor almacenado en la dirección de memoria de datos, o equivalentemente:

rd <= DMem[addr] (donde **addr = rs1 + imm**)

Store (SW,SH,SB). Su formato es el siguiente y son las únicas que usan el estilo de instrucción S:

STORE rs2, imm(rs1)

Estas instrucciones escriben en una dirección específica de la memoria de datos el valor del registro fuente rs2:

DMem[addr] <= rs2 (donde **addr = rs1 + imm**)

Note que, para ambas instrucciones, la dirección desde donde se cargan o almacenan los datos se calcula como **rs1 + imm**, que es el mismo cálculo realizado por la ALU para la instrucción ADDI. Eso significa que para las instrucciones LOAD/STORE se puede aprovechar la ALU para calcular la dirección de memoria DMEM.

Según lo indicado anteriormente, debe incluir los siguientes cambios en su implementación, los cuales se muestran en la Figura 16, en color morado:

- Si la instrucción es carga (**is_load** activa) o almacenamiento (**is_s_instr** activa), la ALU debe hacer una suma entre el registro rs1 y el dato inmediato. Cambie las señales de la ALU para que esto sea posible.
- El bus de direcciones de la memoria de datos DMEM se crea a partir de la salida de la ALU, es decir, **daddr = alu_out**
- El bus de datos de salida (**ddata_out**) se usa sólo en la instrucción STORE y toma el valor del registro apuntado por rs2. Haga el cableado para que **ddata_out** tome dicho valor.
- El bus de datos de entrada (**ddata_in**) se usa sólo en la instrucción LOAD. Este valor se debe almacenar en el register file en el registro apuntado por rd. Si revisa la actual implementación de register file, la salida de la ALU se conecta directamente al register file, pero esto debe cambiar. Si la instrucción es LOAD, debe almacenar en el register file el valor que esté en el bus **ddata_in**. Para otra instrucción debe guardar **alu_out** en el RF. Por lo tanto, debe adicionar un MUX para generar la entrada al register file como muestra la Figura 16.

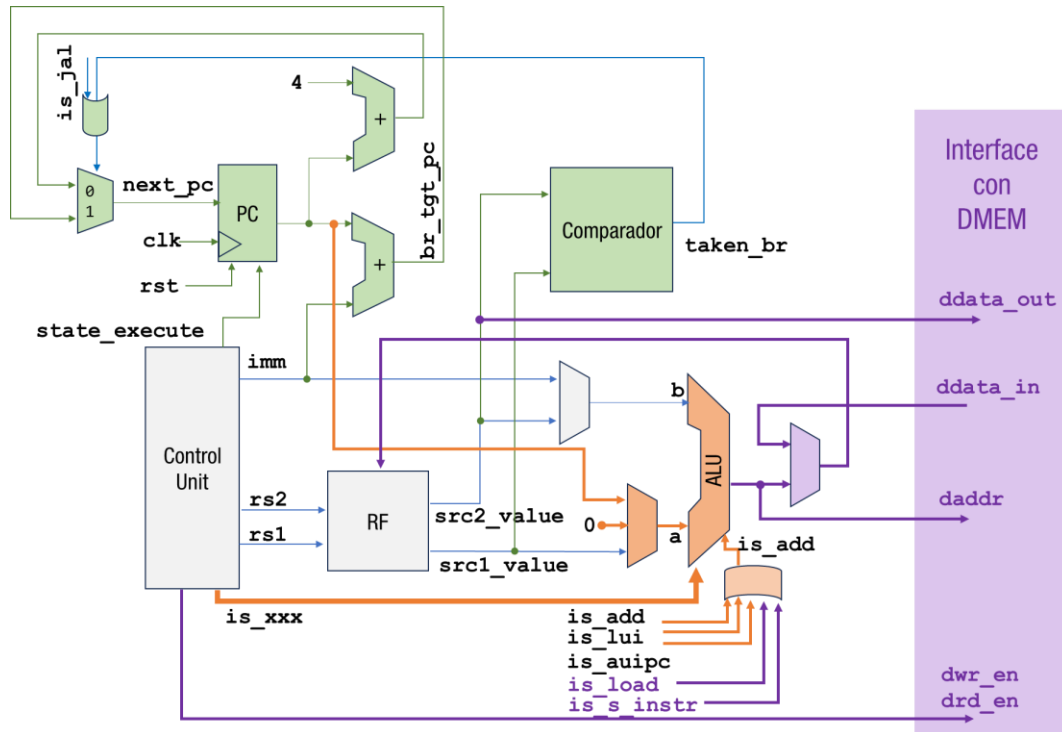


Figura 16. Modificaciones para incluir el soporte de las instrucciones LOAD/STORE en el procesador RISC-V

Además del cableado de la Figura 16, hay que hacer una modificación para la que función LOAD funcione. El problema reside en que algunas memorias SRAM son sincrónicas, por lo que requieren un ciclo de reloj para guardar y entregar los datos, lo cual implica que tengamos que añadir un nuevo ciclo de ejecución en nuestro microprocesador, de lo contrario el dato de entrada al RF no estará listo cuando se ejecuta una instrucción LOAD. En la siguiente tabla se muestra lo que se hace en cada ciclo de reloj:

Ciclo	Implementación actual	Modificaciones al ciclo de instrucción
1 - Fetch	Leer machine code en registro IR	Leer machine code en registro IR
2 - Decode	Generar señales internas para preparar el hardware interno. Leer register file	Generar señales internas para preparar el hardware interno. Leer register file
3 – Execute1	Ejecuta instrucciones ALU Almacena salida ALU en Register File Actualiza PC según condición del Branch	Ejecuta instrucciones ALU Genera dirección para DMEM Activa señales control para DMEM Leer DMEM si la instrucción es LOAD Actualiza PC según condición del Branch
4 – Execute2		Mantiene activas señales control para DMEM Almacena salida MUX en Register File

Debe hacer estas modificaciones en su actual implementación:

- Añada una nueva señal llamada **state_execute2** e inclúyala en el contador en anillo.
- Como el habilitador (enable) del Register File depende de la **state_execute**, cámbielo por la nueva señal **state_execute2**
- Las señales del bus de control para la memoria de datos DMEM, **dwr_en** y **drd_en**, se activan si la instrucción es almacenamiento y carga, respectivamente. Genere estas señales a partir de **is_load** e **is_s_instr**, pero debe condicionarlas a que solamente se pueden activar en ambos ciclos de ejecución (cuando **state_execute** o **state_execute2** estén activos).

Para probar estas nuevas implementaciones, no necesitamos modificar el testbench, solamente es necesario crear el archivo con los códigos de máquina de las instrucciones.

Para automatizar el proceso, vamos a incluir un nuevo paso en nuestro toolchain y es la extracción del código del archivo ELF en formato binario (BIN) y luego la conversión de estos códigos binarios en un archivo de texto HEX que sea compatible con la directiva \$readmemh de Verilog. Para ello debe descargar el programa de Python que encuentra en la página del curso (o usar el comando **hexdump** de la guía anterior) y usar los siguientes comandos:

```
riscv64-unknown-elf-as -o test_program.elf test_load_store.s -march=rv32i
riscv64-unknown-elf-objcopy -O binary test_program.elf test_program.bin
python makehex.py test_program.bin 4096 > test_program.hex
```

Asegúrese que en su archivo de Verilog donde define la memoria de programa IMEM aparezca el mismo nombre HEX de arriba \$readmemh("test_program.hex", mem); El número 4096 en la última línea es porque la memoria de programa es de 4096 palabras.

Para esta prueba, use como código de entrada en el archivo **test_load_store.s** el primer programa en ensamblador al inicio de la guía (Sección 8).

Para verificar el funcionamiento añada las señales en GTKWave que se muestran en la Figura 17.

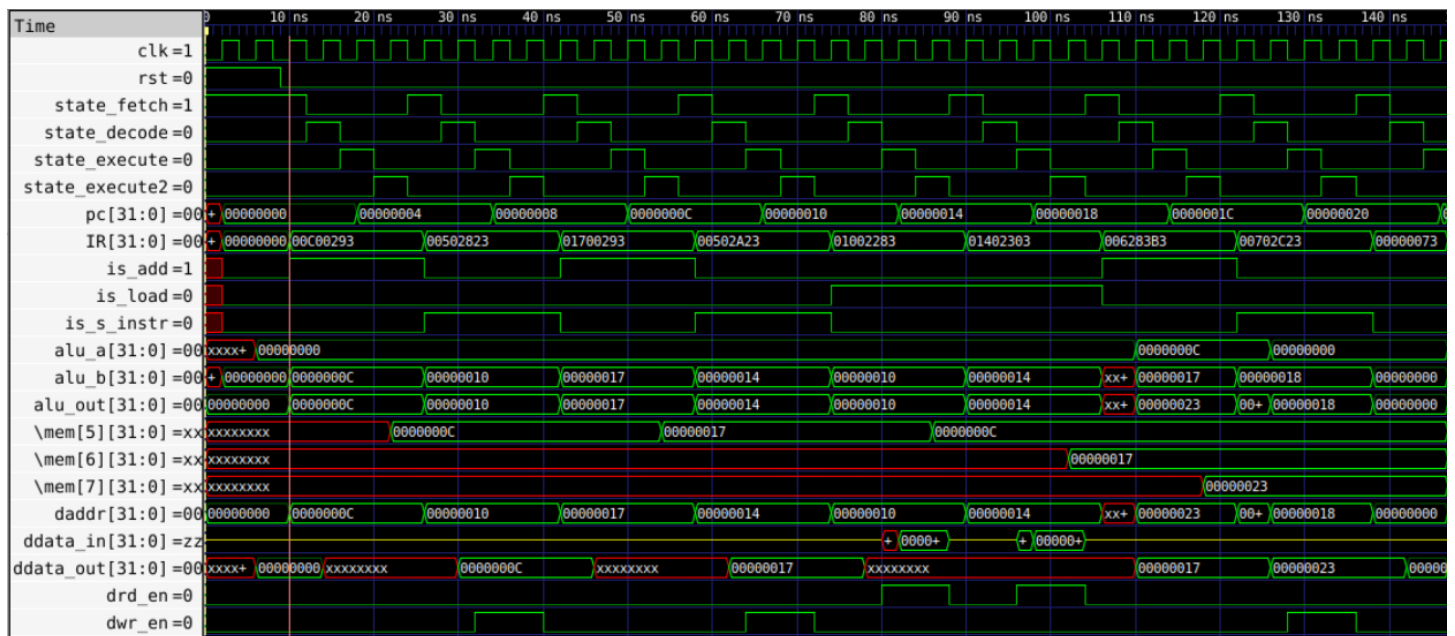


Figura 17. Diagrama de Tiempos para probar el programa de LOAD/STORE